Detecting Code Vulnerabilities using LLMs

Larry Huynh*, Yinghao Zhang*, Djimon Jayasundera*, Woojin Jeon†,
Hyoungshick Kim†, Tingting Bi* and Jin B. Hong*‡
*The University of Western Australia, Perth, Australia
Emails: {larry.huynh, tingting.bi, jin.hong}@uwa.edu.au; {23072214, 23061402}@student.uwa.edu.au
†Sungkyunkwan University, Seoul, South Korea
Emails: {dnwls0116, hyoung}@skku.edu
‡Corresponding author

***Abstract*—Large language models (LLMs) have emerged as a promising tool for detecting code vulnerabilities, potentially offering advantages over traditional rule-based methods. This paper proposes an enhanced framework for vulnerability detection using LLMs, incorporating various prompt engineering strategies to improve performance. We evaluate several techniques, including role-based prompting, zero-shot chain-of-thought, and structured prompting approaches, on the DiverseVul dataset of C/C++ vulnerabilities. Our experiments assess the framework's performance across different code structures, contextual information levels, and LLM capabilities. Our results show that using our dynamic prompt engineering technique, you can improve the F1 score by up to 100% with GPT-3.5, a widely used LLM model. We also observe that GPT-4o, Gemini 2.0 Flash, and Meta Llama 3.1 generally outperform GPT-3.5, and all models are very poor when it comes to correctly identifying the type of vulnerability in the code, with the best F1 score of 0.16 observed. However, our follow-up experiments on LLM-based vulnerability correction (i.e., patching) show a 45.77% success rate using GPT-4o, demonstrating promising results in leveraging LLMs for enhancing software security and providing insights into optimizing prompt engineering for vulnerability detection tasks.**

***Index Terms*—Code Vulnerability, CWE, Large Language Models, Prompt Engineering**

I. Introduction

Software vulnerabilities remain a critical security concern, with early detection during development being crucial for system security, reliability, and robustness. Common vulnerabilities like buffer overflows in C/C++ can lead to severe security breaches if not promptly addressed [1]. Traditional detection methods, including manual code review and rule-based static analysis tools, often prove time-consuming and error-prone [2], [3], highlighting the need for more sophisticated automated solutions. Recent advancements in artificial intelligence, particularly deep learning [4] and other AI techniques [5], offer promising avenues for enhancing vulnerability detection through efficient analysis of vast code repositories and adaptation to emerging threats.

Large Language Models (LLMs) have emerged as powerful tools in software engineering, including code generation, bug fixing, and documentation writing [6]–[8]. Their ability to understand and generate human-like text and code opens new possibilities for automated vulnerability detection. LLMs can generalize from training data to perform well on tasks they weren't explicitly trained for [9] and can identify complex code patterns and structures. However, their application to vulnerability detection remains in early stages, with mixed results. Some studies show LLMs outperforming static code analyzers [10], while others highlight limitations in security concept understanding and high false-positive rates [11], [12], underscoring the need for further research.

Prompt engineering [13] has emerged as a strategy to enhance LLM performance across various tasks, including vulnerability detection [14]. However, while structured prompts can improve response quality, they do not inherently provide models with deeper understanding of code semantics or security-specific contexts. As a result, prompt-based methods alone may struggle with vulnerabilities that require nuanced reasoning beyond pattern recognition. To address this limitation, integrating prompt engineering with context-aware analysis is essential for improving detection performance, particularly when vulnerabilities span multiple functions or rely on implicit program behaviors.

Existing studies on LLM-based vulnerability detection often rely on datasets with isolated vulnerable snippets, which simplifies the detection process. However, in practice, vulnerabilities are embedded within larger code functions or contexts with unknown locations, creating a gap between current datasets and real-world scenarios. Addressing this gap requires methods capable of handling vulnerabilities across varying code contexts, including entire functions or multi-function structures.

In this paper, we propose an enhanced framework for code vulnerability detection (CVD) using LLMs with prompt engineering strategies. Our approach addresses current LLM limitations through carefully crafted prompts and context-aware analysis. To investigate how LLMs detect code vulnerabilities, we focus on two critical dimensions: (1) prompt engineering—what we say when instructing the LLM, and (2) context—what and how much information we provide about the code. We compare various prompting strategies (e.g., chain-of-thought, role-based prompts) and contextual information levels (e.g., snippet-only vs. full-file) on the DiverseVul dataset [15] of C/C++ vulnerabilities.

We address the following research questions:

- **RQ1**: Do different prompt engineering strategies affect the performance of vulnerability detection using LLMs?

- **RQ2**: Do the capabilities of different LLM versions influence vulnerability detection accuracy?
- **RQ3**: What is the impact of varying levels of code context on detection performance?
- **RQ4**: Can LLMs effectively detect vulnerabilities found in single-function and multi-function code structures?

Our contributions include:

- A novel framework for LLM-based vulnerability detection, incorporating six prompt engineering techniques and their comprehensive evaluation
- Analysis of code context and LLM capabilities on detection performance, including comparisons between top 25 Common Weaknesses Enumeration (CWE) vulnerabilities and others
- Insights into LLMs' effectiveness in handling vulnerabilities embedded within full functions and varying code contexts, addressing the gap between current datasets and real-world scenarios
- Identification of challenges and opportunities in using LLMs for both vulnerability detection and correction

Our work advances automated vulnerability detection and provides insights for researchers, practitioners, and the general public using LLMs for coding projects.

The remainder of this paper is organized as follows. Section II describes our methodology, Section III presents results, Section IV discusses implications, Section V addresses validity threats, Section VI reviews related works, and Section VII concludes with future directions.

II. Methodology

We present an approach to CVD that leverages LLMs and various prompt-based engineering strategies. The core of our framework involves providing the LLM with carefully crafted prompts containing potentially vulnerable code, accompanied by contextual information such as comments and docstrings. By combining prompt engineering with context-aware analysis, our approach ensures that LLMs not only leverage structured instruction but also interpret vulnerabilities within the broader scope of real-world code. This enables a more nuanced assessment beyond pattern matching, improving the model's ability to detect complex vulnerabilities spanning multiple functions or implicit logic. The framework capturing the workflow is shown in Figure 1. This adaptive approach allows for continuous refinement of the detection process, ensuring that the system's accuracy and robustness improve over time. The code is also provided on GitHub[1].

Before populating contents for each code vulnerability detection phase, we conducted a pre-study survey to understand user perceptions and preferred approaches currently taken (as shown in Section II-A). Then, we present the dataset used in Section II-B. The details of prompt engineering techniques are found in Section II-C. Using the dataset, prompt engineering techniques, and the selected LLMs, we describe the code vulnerability detection in Section II-D. The evaluation criteria and metrics used are shown in Section II-E.

[1]https://github.com/a24167566/LLMs-Code-Vulnerability-Detection.git

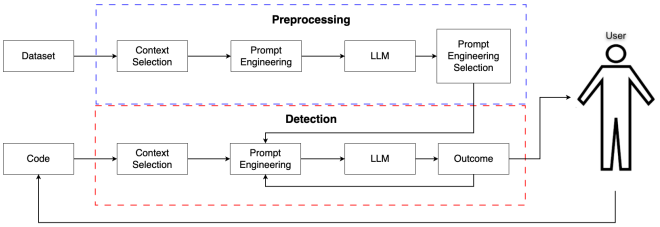


Fig. 1: Overview of the CVD Framework.

A. Pre-study: A User Study

We conducted an online survey via Prolific to investigate attitudes towards LLMs in secure coding. The survey targeted respondents from English-speaking countries (US, UK, Canada, and Australia) to ensure linguistic consistency with the primarily English-trained LLM models [16], thus mitigating potential language-related biases.

To ensure data quality and relevance, we implemented a two-step validation process: screening participants for LLM programming experience and requiring a validation code upon survey completion. This rigorous approach yielded 105 valid participants, providing 1,470 unique responses across 14 questions. The survey had a median completion time of 5 minutes, with participants compensated at a flat rate of $2.50 USD. Our study strictly focused on the use of LLM in programming contexts, adhering to ethical research practices by collecting only necessary information and protecting participant privacy.

The questions themselves span three main categories: (1) participant details, (2) the general use of LLMs for coding, and (3) the use of LLMs for secure coding practices. Questions assessed participants' frequency of LLM use for code assistance (from 0% to 100% of coding time), as well as their confidence in and satisfaction with LLM-generated code, both generally and specifically regarding secure code writing, identification, and correction. We also explored participants' prompting strategies for secure code writing and review. Respondents could select multiple approaches from a list, including zero-shot, few-shot, chain-of-thought, recursive criticism and improvement, security expert persona, explicit security requirements, and vulnerability-aware prompting. The complete set of questions and anonymized responses are available on our GitHub.

The key survey questions (SQ) asked are summarized below.

- **SQ1**: Which LLM do you prefer for coding assistance?
- **SQ2**: How satisfied are you with code written by LLMs for your programming tasks?
- **SQ3**: How often do you consider the security of the code you write?
- **SQ4**: How much do you trust that the code written by the LLM is secure?

The survey results guide our selection of LLM models and prompt strategies for evaluation, ensuring our research aligns

TABLE I: LLM Secure Coding Pre-Survey; Key Results.

Question	Never	Occasionally	Frequently	Always
SQ2	2.9%	46.6%	48.6%	1.9%
SQ3	8.6%	41.9%	25.7%	23.8%
SQ4	18.1%	48.6%	28.6%	4.7%

with real-world behaviors in secure coding. For SQ1, our survey shows that 58.1% of participants preferred using OpenAI's GPT series, with GPT-4 models being the most popular with 48.6%. The next most popular vendor was Microsoft with the next highest proportions being 10.5% for Copilot, followed by 9.5% for OpenAI GPT-3.5. GPT-4o has also been noted to be more performant than GPT-4 over a variety of benchmark tasks [17]. The next preferred models were Claude 3.5 Sonnet (8.7%) and Google Gemini (7.7%). We include Gemini Flash 2.0 as Gemini models are comparatively popular to Claude 3.5 Sonnet, but is available at approximately 30x reduced API cost ($3 vs $0.1 per million input tokens; Claude vs. Gemini), making it an attractive alternative to currently available services[2]. The next provider afterwards is Meta with Llama 3(1.9%). Despite its lower usage, we include this model as it offers a free, locally-deployable option valuable for secure coding applications requiring data privacy. We thus conduct experiments using GPT-4o, GPT-3.5 (as a low-cost alternative to GPT-4o), Google Gemini Flash 2.0, and Meta's Llama 3. Unfortunately, there is a lack of accessible API for Microsoft Copilot, so it was omitted in our experiments.

Table I summarizes the response for each survey question. Various observations are made that are critical to users when it comes to LLMs and secure coding. First, users are generally over-reliant on LLMs, where only 3% of participants indicated that they stopped using an LLM for coding after experiencing it at least once. This means over 97% are using LLMs to assist with code writing. Secondly, users generally trust the code outputs provided by LLMs in both general and secure coding contexts. An overwhelming 91.4% of participants are concerned with the security of the code they write (i.e., SQ3), but 81.9% trust the security of LLM-generated code to some degree (i.e., SQ4). This implies that users have a high level of confidence in LLMs for secure coding tasks. Furthermore, the survey reveals a notable difference in how users perceive the capabilities of GPT-3.5 and GPT-4o in secure coding assistance. Among GPT-3.5 users, only 10% reported trusting its capabilities "Frequently" or "Always." In contrast, 38.1% of GPT-4o users expressed this level of trust. This may be because participants generally view GPT-4o as more capable than GPT-3.5, suggesting a higher level of trust in more advanced models for security-related tasks.

However, this reliance and trust in LLMs for secure coding contrasts with findings from previous studies, which have shown that the performance of LLMs in secure code assistance is highly varied with some results indicating significant shortcomings [11], [12]. This discrepancy between user perception and actual LLM performance in secure coding tasks highlights a potential risk: users may be overestimating the security capabilities of these models, potentially leading to the introduction of vulnerabilities in their code.

In terms of techniques used with LLMs by users, we observe that the chain-of-thought prompting technique was most popular for both writing secure code and reviewing code security, preferred by 52.4% and 43.8% of participants, respectively. The other prompting strategies were almost equally preferred. Thus, we also investigate the effect of different prompting techniques and their effects on secure coding, which are further detailed in Section II-C.

[2]API pricing for Claude: https://www.anthropic.com/pricing#anthropic-api and Gemini models: https://ai.google.dev/gemini-api/docs/pricing

TABLE II: Summary Statistics of Datasets Considered.

Dataset	Projects	CWEs	Functions	Labels	GitHub Link
Devign [19]	2	N/A	26037	No	Yes
BigVul [20]	348	91	264919	Yes	Yes
CrossVul [21]	498	107	134126	Yes	No
CVEFixes [22]	564	127	168089	Yes	Yes
DiverseVul [15]	797	150	330492	Yes	Yes

B. Dataset

To validate our method, we utilized the DiverseVul dataset [15], a comprehensive aggregation of existing C/C++ vulnerability datasets sourced from various GitHub projects. Within this dataset, each datapoint corresponds to a specific GitHub commit designed to address one or more identified code vulnerabilities, each classified by their CWE code [18].

We use the DiverseVul dataset rather than other similar options, shown in Table II, for the following reasons:

- **Comprehensiveness**: It aggregates vulnerabilities in 330,492 functions from 797 GitHub projects, encompassing a wider range of code patterns and vulnerability types compared to individual datasets like ReVeal or BigVul.
- **Representation**: The dataset captures a diverse range of 150 CWEs, with many of the most common CWE labels being present. Further, there is a strong overlap with the CWEs contained in MITRE's Top 25[3], including CWE-787 (no. 1), CWE-416 (no. 4), CWE-20 (no. 6), and CWE-125 (no. 7).
- **Real-World Relevance**: The strong representation of common and critical CWEs enhances the dataset's relevance for real-world vulnerability detection tasks. Further, unlike synthetic datasets such as SARD, DiverseVul sources vulnerabilities from actual GitHub commits, ensuring their relevance to practical development.
- **Recency**: Published in 2023, DiverseVul is a relatively recent dataset compared to others like Devign (2019) or CVEFixes (2021). This recency captures the latest vulnerability patterns and trends, making it more applicable to current software development practices.
- **Full Context Availability**: Importantly, DiverseVul allows us to retrieve the full code context for each vul-

[3]https://cwe.mitre.org/top25/

nerability. In practice, developers analyze code within its broader context, not in isolation. Most existing datasets do not offer the ability to reconstruct full code context from raw data. DiverseVul, however, retains the necessary commit-level information, allowing us to link each vulnerable function to its surrounding code. This is essential for evaluating LLMs under conditions that reflect real-world usage, as vulnerabilities often span multiple code locations, and evaluating only on isolated snippets would not capture the complete vulnerability landscape.

Other datasets, such as Devign, BigVul, and CrossVul, primarily contain pre-extracted code snippets targeting specific vulnerable functions without broader context. While valuable for analyzing single-function vulnerabilities, they lack the full context needed to evaluate LLM performance in realistic scenarios. If a developer has already pinpointed the exact vulnerable function, they likely have enough information to address it without an LLM's help. This makes DiverseVul the only suitable dataset for our analysis, as other datasets do not provide the code context required to emulate real-world vulnerability detection.

To enhance the utility of the dataset, we apply several augmentation steps:

- **Full Context Retrieval:** Leveraging GitHub's API, we systematically link each vulnerability-fixing commit to its associated source code files. This ensures that the models have access to the complete context surrounding each vulnerability.
- **Data Normalization:** To optimize data organization and eliminate redundancy, we transform the original flat structure of the dataset into a hierarchical format.

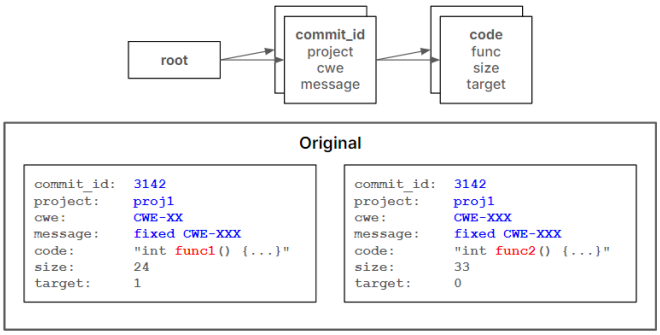


Fig. 2: Structure of DiverseVul Dataset.

We further refine the dataset through a series of post-processing operations. We separate data points into vulnerabilities that existed across multiple functions and those that were contained within a single function. We also remove data points lacking a CWE identifier or containing multiple CWEs. We also remove commits that changed more than one file to ensure all functions examined are relevant to the CWE identifier.

We then apply our prompt engineering strategies (Section II-C) to the dataset to investigate the capabilities of each strategy in enhancing the performance of LLM-based code vulnerability detection. Each data point (or code input) is integrated with the relevant prompt strategy, producing six (one baseline and five prompt engineering) versions of the DiverseVul dataset.

C. Prompt Engineering Strategies

We explore several prompt engineering techniques to enhance the performance of LLMs in detecting code that leads to software vulnerabilities (which we will denote as code vulnerabilities). These techniques are designed to improve the model's reasoning capabilities and the quality of its outputs. The following prompt engineering strategies were implemented and evaluated:

- **Baseline:** The LLM is given the affected code as its only input - no prompt engineering is applied here. This serves as a baseline for the other prompt engineering strategies. Example Baseline prompt:

  Analyze the provided code and determine if the code is vulnerable or safe, if vulnerable identify the most obvious CWE.
  [CODE]
- **Role-based Prompting:** The LLM is instructed to assume the role of a cybersecurity expert. This approach has demonstrated superior performance compared to Chain-of-Thought (CoT) and zero-shot CoT in certain reasoning tasks, while still eliciting CoT-like behavior in the LLM's responses [23]. Example Role-based prompt:

  From now on, you are an excellent cybersecurity expert who can analyze given code and determine if it is vulnerable or safe, and if vulnerable, identify the related CWE.
  [Baseline prompt]
- **Zero-shot Chain-of-Thought (CoT):** This strategy enhances the prompt with the phrase "Let's think step by step," encouraging the LLM to outline its reasoning in stages [24]. This approach guides the model to engage in sequential reasoning, which has been shown to improve accuracy in tasks requiring logical deduction without needing few-shot exemplars. Example Zero-shot CoT prompt:

  [Baseline prompt]
  Let's think step by step.
- **APE Zero-shot CoT:** APE (Automatic Prompt Engineering) zero-shot CoT builds upon the zero-shot CoT method with a refined prompt that encourages a more structured reasoning process. The APE framework automates prompt selection by iterating through options and assessing outcomes across tasks; in their original study, the authors identified "Let's work this out in a step-by-step way to be sure we have the right answer" as the optimal prefix for complex reasoning tasks using their APE-CoT strategy [25]. This prompt was then shown to outperform both standard zero-shot CoT and human-crafted CoT prompts across various reasoning tasks. In our study, APE zero-shot CoT was selected to assess

whether automated prompt refinements could enhance the LLM's consistency in detecting code vulnerabilities. Example APE-CoT prompt:

[Baseline prompt]
Let's work this out in a step-by-step way to be sure we have the right answer.

- **CO-STAR:** The prompt follows a defined structure that uses XML tags to define Context, Objective, Style, Tone, Audience, and Response [26]. This technique has proven effective in improving LLMs' content generation capabilities across various tasks. Example CO-STAR prompt:

  <CONTEXT>
  You are asked to accurately detect if a code is vulnerable or safe. The vulnerabilities are related to the Common Weakness Enumeration (CWE) list.
  </CONTEXT>
  <OBJECTIVE>
  [Baseline prompt]
  </OBJECTIVE>
  <STYLE>
  Provide answers based on the Common Weakness Enumeration descriptions and IDs.
  </STYLE>
  <TONE>
  Logical.
  </TONE>
  <AUDIENCE>
  Your response will be read by cybersecurity experts to assess the accuracy of your response.
  </AUDIENCE>
  <RESPONSE>
  Analysis of the code and your determination of whether the code is vulnerable or safe. If the code is vulnerable, include the most obvious CWE in your analysis.
  </RESPONSE>

- **Dynamic:** We propose a dynamic prompting technique that allows the LLM to analyze functionality and predict potential vulnerabilities by passing the prediction prompt once. This method constructs dynamic prompts using XML tags with Context, Objective, and Hint in a structured format inspired by [26]. Our approach offers a minimal, two-step alternative to fully iterative multi-round prompting: first, the model provides a concise "preview" or high-level inference about potential vulnerabilities, then this inference is embedded into a follow-up prompt as a "hint." While loosely reflecting iterative prompting, this technique avoids lengthy back-and-forth exchanges—remaining closer to zero-shot usage by leveraging only the model's own output alongside the original input code. The design captures advantages of iterative methods (such as clarifying ambiguous code snippets) without the added complexity of multiple user interactions or specialized research workflows. We hypothesize that incorporating such extra dynamic auxiliary information in a structured format can help the LLM improve its vulnerability analysis.
  Example Prediction prompt:

  Analyze the provided code below and return two lines of text. The first line states the programming language of the code. The second line is a list of potential vulnerabilities based on the code's functionality ONLY. Do not return any code.
  Code to analyze: [CODE]

  Example Dynamic prompt:

  <CONTEXT>
  You are asked to accurately detect if a code is vulnerable or safe. The vulnerabilities are related to the CWE list.
  </CONTEXT>
  <OBJECTIVE>
  [Baseline prompt]
  </OBJECTIVE>
  <HINT>
  Ensure your analysis is accurate. Use the following information to assist your analysis:
  [Prediction response]
  </HINT>

These prompting techniques are selected due to their zero-shot property. Unlike other prompting techniques, which require a few-shot approach, these techniques can evaluate LLM's performance and the influence of prompting without extra information besides the given code. Additionally, few-shot approaches require a set of vulnerability detection examples. Given that our dataset only contains code and CWE pairs, such data would not serve as good examples due to the lack of vulnerability analysis. Due to this limitation, few-shot approaches are not considered in this paper. Further, our user study revealed that participants far less preferred recursive/iterative approaches (9.48% of participants) to chain-of-thought and zero-shot prompting methods (19.83% and 13.79% respectively), so we did not explore automated prompt-search frameworks like DSPY [27] that require repeated interaction loops. By focusing on readily adoptable strategies, we aim to reflect typical developer usage patterns rather than specialized research workflows.

D. Code Vulnerability Detection

Our code vulnerability detection framework leverages LLMs as its core component. We conducted experiments using OpenAI's gpt-3.5-turbo-0125 (referred to as GPT-3.5), gpt-4o-2024-05-13 (GPT-4o), Google's gemini-2.0-flash-001 (Gemini 2.0 Flash), and Meta's Llama 3.1 8B (Llama 3.1). While our approach is fundamentally LLM-agnostic, we selected these specific models based on the results of our user survey shown in Section II-A. For these experiments, we used the models' default temperature setting ($t = 1$), as this setting aligns with typical user interactions. While this may introduce some variability in responses, studies suggest that even with this non-deterministic setting, acceptable results are minimally

impacted, as outputs tend to remain contextually consistent [28], [29].

For each dataset version (from Section II-B) and LLM, we query the model and store its responses. These responses include the full prompt used, the labeled CWE (or "SAFE" for non-vulnerable code), and the LLM's complete output. Our response analysis process incorporates a cleaning layer that performs keyword searches for identified CWEs or patterns indicating the LLM's assessment of code safety. After cleaning, we categorize the response as either a specific CWE or "SAFE." Responses marked as "Error" or those lacking a clear pattern are considered failed responses and excluded from correctness evaluation.

E. Evaluation

We devise a set of experiments to assess the effectiveness of the detection framework. These experiments aim to evaluate the framework's performance across different code structures, prompt strategies, contextual information levels, and LLM capabilities. The variables and metrics used to evaluate the detection performance of LLMs are described here.

1) Variables: We examine five variables in our experiments to gain an in-depth understanding of the use of LLMs to detect code vulnerabilities. They are (1) prompt engineering strategies, (2) the varying levels of context provided by the code, (3) the scope of vulnerability in single or multiple functions, (4) different capabilities of LLMs, and (5) the objective of LLMs for detecting vulnerable code whether it would be fully automated or involving a "human-in-the-loop". Further details are provided below.

1. Prompt Engineering Strategies: We separately apply each of the prompt engineering strategies (Section II-C) to the vulnerable code artifacts in the dataset (Section II-B). Every data point is thus given to the LLM for detection, each testing the six prompt engineering strategies.

2. Effect of Context in Detection Performance: Additionally, we generate three versions of the DiverseVul dataset, with each successive version containing more context about the affected code (data point):

- **No Context (NC):** Only the code snippet containing the secure or vulnerable code is given to the LLM.
- **Semantic Context (SC):** The code snippet and any accompanying docstrings or code comments are also included.
- **Full Context (FC):** The entire file that contains the code snippet (including docstrings and comments) is given to the LLM.

We deliberately scope our context analysis to single-file boundaries for several reasons. First, this approach aligns with typical developer workflows, where vulnerability checks are often performed file-by-file. Second, it respects the practical context window limitations of current LLMs. While multi-file or repository-wide analysis could potentially provide additional signals, such contexts may exceed model limitations and increase user costs. Furthermore, vulnerability remediation often requires fixing only a single component in a chain, which can be identified through single-file analysis. We consider multi-file vulnerability detection a valuable direction for future work.

TABLE III: Classification of Code Vulnerability.

	Benign classification	Vulnerable classification	Mis-classification
Benign code	True Negative (TN)	False Positive (FP)	-
Vulnerable code	False Negative (FN)	True Positive (TP)	Mis-classification (MC)

3. Single Function Vs. Multiple Function Vulnerabilities: We perform the following experiments on two separate subsets of our dataset: Vulnerable code that is expressed in a single function, and vulnerable code that is expressed across multiple functions. This distinction allows us to assess the framework's ability to detect vulnerabilities in varying code complexities.

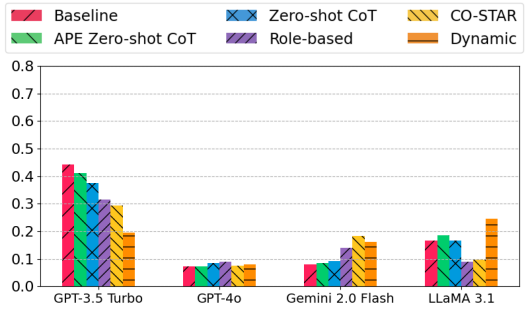
4. Using Different LLMs for the Detection Task: As mentioned in Section II-D, we test the capabilities of four LLMs: GPT-3.5, GPT-4o, Gemini 2.0 Flash, and Meta Llama 3.1. We hypothesize that models with higher capabilities, such as GPT-4o and Gemini 2.0 Flash, would (1) benefit further from the inclusion of greater context, and (2) better handle the more complex task of classifying multiple function vulnerabilities.

5. Fully Automatic and Human-in-the-Loop Approaches: To investigate the potential use of LLMs as code vulnerability detection tools from different perspectives, we consider both fully Automatic and Human-in-the-Loop (HITL) approaches. In the Automatic approach, we assess the LLM's ability to not only detect vulnerabilities but also accurately identify the CWE type. In contrast, the Human-in-the-Loop is presented as a proposed context; the approach prioritizes detecting whether code is vulnerable or safe, delegating CWE identification and further analysis to a (hypothetical) human reviewer.

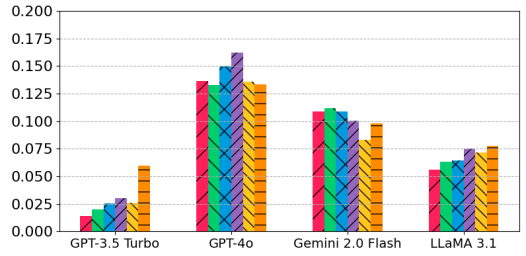
2) Performance Metrics: There are five outcomes of using LLMs for detecting vulnerable code, shown in Table III. In the case of TP, the LLM correctly identifies the CWE associated with the vulnerability in the code. In the case of MC, the LLM flags the code as vulnerable but misclassifies the CWE type. Since benign code has no associated vulnerability, MC does not apply to benign cases. In the Automatic approach, the objective is fully autonomous vulnerability detection and correction. Here, MC is treated as a misclassification, as incorrect CWE identification could result in inappropriate corrective actions. Precise CWE identification is thus essential for enabling an automated end-to-end response. In the Human-in-the-Loop approach, by contrast, the LLM's role is limited to vulnerability detection, while a human expert reviews and verifies flagged vulnerabilities. Consequently, MC is treated as a correct classification, as identifying the code as vulnerable — irrespective of the specific CWE — alerts the reviewer to investigate further. Including MC as a correct classification aligns with this approach's objective, where detection alone suffices to meet the LLM's role.

The metrics are calculated according to these objectives, as detailed below:

Case 1: Automatic approach. In this scenario, the LLM must correctly identify the CWE for an automated response.

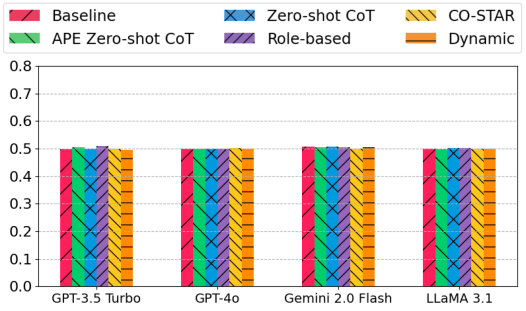


(a) Accuracy

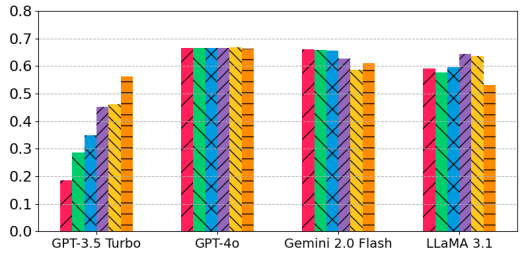


(b) F1 score

Fig. 3: Automatic Approach: Performances of Different Prompt Engineering Techniques.



(a) Accuracy



(b) F1 score

Fig. 4: Human-in-the-Loop (HITL) Approach: Performances of Different Prompt Engineering Techniques.

MC is therefore counted as an incorrect outcome, as an inaccurate CWE could lead to an ineffective or inappropriate fix.

Accuracy	:	(TP + TN) / (TP + TN + FP + FN + MC)
Precision	:	TP / (TP + FP)
Recall	:	TP / (TP + FN + MC)
F1 score	:	2 * (Precision * Recall) / (Precision + Recall)

Case 2: Human-in-the-Loop approach. The primary goal here is identifying potentially vulnerable code, while exact CWE classification is secondary. Consequently, MC is considered a correct outcome, as it signals potential vulnerability and prompts further review by an expert.

Accuracy	:	(TP + TN + MC) / (TP + TN + FP + FN + MC)
Precision	:	(TP + MC) / (TP + FP + MC)
Recall	:	(TP + MC) / (TP + FN + MC)
F1 score	:	2 * (Precision * Recall) / (Precision + Recall)

III. Results

Our experimental results with respect to the variables described in Section II-E are shown here. In particular, we show the prompt engineering techniques and their effects on code vulnerability detection in Section III-A, which answers RQ1 and RQ2. For this section, the whole dataset is used for each prompt engineering technique. Then, data and vulnerability detection with respect to code context, single and multiple files, and CWE are shown in Section III-B, answering RQ3 and RQ4. For this section, all prompt engineering techniques are used, and their results are averaged with respect to each context. For the experiment setup, we used a 2022 MacBook Air with an Apple M2 chip and 16 GB of RAM. We accessed GPT-3.5 and GPT-4o through OpenAI's API[4], and Gemini through Google's API[5]. The LLM-based detection experiments thus utilized OpenAI's cloud resources for generation.

A. Prompt Engineering and Vulnerability Detection

RQ1: Do different prompt engineering strategies affect the performance of vulnerability detection using LLMs?

Answer: The results from both Figures 3 and 4 demonstrate that different prompt engineering strategies significantly impact the performance of vulnerability detection using LLMs. Figure 3 illustrates variations in accuracy and F1 scores across different prompt engineering techniques (Baseline, APE Zero-shot CoT, Zero-shot CoT, Role, CO-STAR, Dynamic) for our LLMs. For GPT-3.5, the Baseline strategy yielded higher accuracy but a lower F1 score compared to other strategies, while the inverse is true for the Dynamic strategy. This suggests that the Dynamic strategy is most capable of detecting the correct CWE. A similar assessment can be made for Gemini 2.0 Flash, but for the Baseline, APE Zero-shot CoT, and Zero-shot CoT methods. For GPT-4o, the Role-based prompting strategy showed the strongest performance compared to other prompt engineering methods, while Llama 3.1 showed strong performance in both accuracy and F1 for Dynamic, reinforcing its capabilities. Figure 4, however, presents a slightly different picture. For GPT-3.5, the dynamic prompting method scores best in the F1 score, with accuracies for all strategies being very similar. For GPT-4o, there does not seem to be a strong bias towards any given strategy. In both Gemini 2.0 Flash and Llama 3.1, Dynamic prompting demonstrates a slightly

[4]https://platform.openai.com/docs/overview
[5]https://ai.google.dev/gemini-api/docs/models/gemini

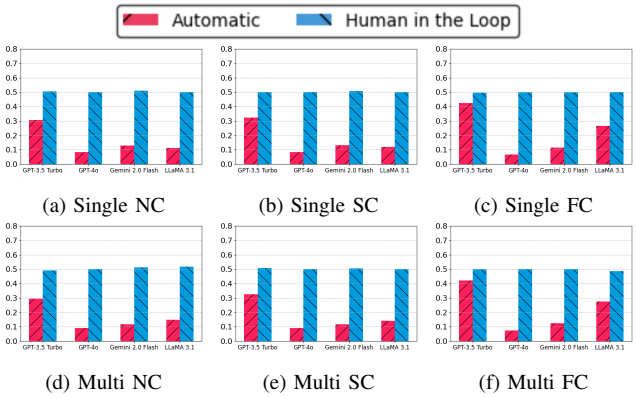
weaker F1 score comparatively, but with higher accuracy than the aforementioned methods, suggesting that Dynamic prompting improves detection but is less effective for CWE-specific identification in these models.

These results indicate that prompt engineering can significantly influence the vulnerability detection performance of specific LLMs. Notably, our proposed Dynamic method increased GPT-3.5's F1 score, reflecting the model's improved performance at identifying the correct CWE. However, with GPT-4o, the prompt engineering techniques did not influence the results as substantially. This difference in impact could be attributed to several factors. One plausible explanation is that the larger token sizes of GPT-4o and Gemini 2.0 Flash may overcome the limitations addressed by more sophisticated prompt engineering techniques. Consequently, when using LLMs with limited token sizes, we can expect improved performance through the application of advanced prompt engineering techniques. In contrast, for LLMs capable of processing much larger token sizes, the importance of specific prompt engineering techniques appears to diminish. Furthermore, the impact of prompt engineering could potentially decrease inversely to model capabilities, as expressed through larger model sizes, more extensive training datasets, and longer training times. Additional analysis and research may help elucidate these relationships and provide deeper insights into optimizing prompt engineering techniques across different LLMs.

RQ2: Do the capabilities of different LLM versions influence vulnerability detection accuracy?

Answer: Analysis of Figures 3, 4, 5, and 6 reveals distinct performance characteristics for our four models in vulnerability detection tasks. In the automatic approach, GPT-3.5 demonstrates higher accuracies but lower F1 scores compared to the other models across various prompt engineering techniques. This suggests that while GPT-3.5 is better at identifying the presence of vulnerabilities, the other models are more precise in classifying the specific type of vulnerability, with GPT-4o being the best, then Gemini 2.0 flash, then Llama 3.1. In the human-in-the-loop approach, all models show consistent accuracies of approximately 50%. However, their F1 score performance differs significantly. GPT-3.5 exhibits degraded F1 scores, with the Dynamic prompting technique achieving approximately 55%. In contrast, GPT-4o and, to a lesser extent, Gemini 2.0 Flash and Llama 3.1 maintain both stronger (approximately 67%, 63%, and 59% on average, respectively) and more consistent F1 scores across different prompting strategies.

This discrepancy highlights a fundamental trade-off between accuracy and F1 score in vulnerability detection. GPT-3.5's higher accuracy but lower F1 score suggests a bias toward "safe" classifications, reducing vulnerability recall. In contrast, other models achieve higher F1 scores by correctly identifying more vulnerabilities, despite generating more false positives. Since missing vulnerabilities poses a greater risk than false alarms, the higher-F1 models (GPT-4o, Gemini, and Llama) are preferable for security applications. However, the low F1



(a) Single NC (b) Single SC (c) Single FC

(d) Multi NC (e) Multi SC (f) Multi FC

Fig. 5: Accuracy comparison between Automatic and Human-in-the-Loop approaches under different contexts.

scores across all models indicate that even the best-performing configurations struggle with vulnerability identification. This suggests that fundamental limitations in LLM-based detection persist, further reinforcing the need for verification mechanisms or hybrid approaches with human oversight.

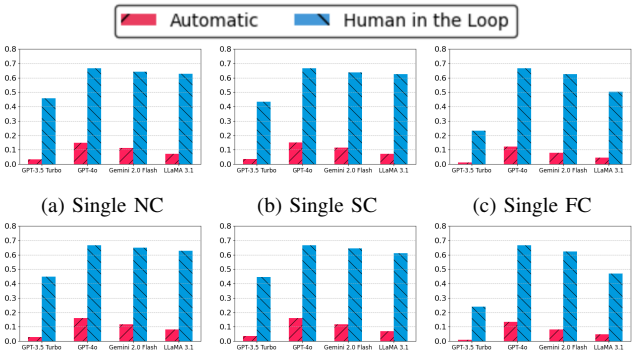
These findings have implications when developing end-to-end frameworks using LLMs for both automatic detection and human-in-the-loop scenarios. GPT-4o's higher F1 score indicates more frequent correct CWE identification in automatic detection, while GPT-3.5 using the Dynamic prompting strategy may be a viable, lower-cost alternative in human contexts, where flagging potential vulnerabilities is prioritized over correct CWE classification. Both Gemini 2.0 Flash and Llama 3.1 offer compelling alternatives, delivering performance comparable to GPT-4o at significantly lower costs - Gemini as a newer, more affordable model and Llama as a free, locally-deployable option.

While the human-in-the-loop approach is valuable for leveraging expertise and improving detection accuracy, it may not suffice for end-to-end automated pipelines with correction mechanisms. In such systems, LLMs are used to fix the vulnerable code once it is flagged by the detection components. If the framework flags the wrong CWE, it is less likely that the vulnerable code will be correctly fixed. In this context, GPT-4o's more precise CWE identification could lead to more appropriate fixes in automated correction pipelines.

B. Data and Vulnerability Detection

RQ3: What is the impact of varying levels of code context on detection performance? and **RQ4**: Can LLMs effectively detect vulnerabilities found in single-function and multi-function code structures?

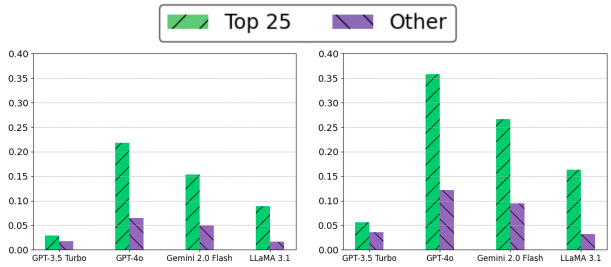
Answer: The results shown in Figures 5 and 6 show that the accuracy of detecting code vulnerabilities with respect to the code context varies if we consider the automatic approach (i.e., correctly identifying the CWE), but the performance is about the same if we consider the human-in-the-loop approach (i.e., the vulnerable code is identified regardless of correct or incorrect CWE).



(a) Single NC (b) Single SC (c) Single FC

(d) Multi NC (e) Multi SC (f) Multi FC

Fig. 6: F1 score comparison between Automatic and Human-in-the-Loop approaches under different contexts.



(a) Accuracy (b) F1 score

Fig. 7: LLM Performance for Top 25 and Remaining CWEs.

A comparison of Figures 5 and 6 reveals that GPT-3.5 generally achieves higher accuracy but lower F1 scores than both GPT-4o and Gemini 2.0 Flash for both automatic and human-in-the-loop approaches. Llama 3.1 shows a similar pattern of higher accuracy but lower F1 scores when compared to GPT-4o and Gemini 2.0 Flash, though this disparity is less pronounced than with GPT-3.5. This suggests that GPT-3.5 is more effective in identifying the presence of vulnerabilities in a general sense, even when it misclassifies specific CWEs or produces false positives, leading to a lower F1 score. In contrast, GPT-4o's higher F1 score reflects its superior ability to correctly classify specific CWEs, even if it occasionally misses vulnerabilities, resulting in slightly lower overall accuracy. Gemini 2.0 Flash follows a similar pattern, with marginally higher accuracy and lower F1 scores than GPT-4o across all examined contexts. This trade-off suggests that GPT-3.5 and Llama 3.1 prioritize breadth of detection, while GPT-4o and Gemini 2.0 Flash emphasize precision in vulnerability classification. With regard to vulnerabilities present in single or multiple functions, the performance observed was the same for both models. This indicates that even with LLMs capable of processing larger tokens (and hence having a better contextual understanding of the code), they are still unable to improve the detection of vulnerabilities spanning multiple functions. However, the detection rate is comparable to that for single-function vulnerabilities, suggesting that these models tend to detect any code that resembles a vulnerability. While this ensures detection parity across single- and multi-function contexts, the models have limited capability to explain such vulnerabilities without fully understanding their broader scope. Furthermore, both GPT-4o and Gemini 2.0 Flash's F1 performance remains stable as the code context grows, benefiting from its larger context window, whereas GPT-3.5 experiences significant degradation as the context increases. This highlights former two model's advantage in processing extensive information effectively.

We also analyzed whether detection performance varied based on vulnerability prevalence. Figure 7 illustrates the comparative performance in detecting the top 25 CWEs versus less common vulnerabilities. Results demonstrate that all models achieved substantially higher accuracy and F1 scores when identifying frequently discussed vulnerabilities compared to less common ones. This correlation between detection capability and presumed training data frequency suggests that model performance is proportional to vulnerability exposure during training. The performance disparity between these categories is substantial, where up to 3.5 times better performance is observed (e.g., F1 score using GPT-4o). This finding identifies a potential security concern: adversaries could circumvent LLM-based vulnerability detection systems by deliberately exploiting uncommon vulnerabilities, thereby introducing insecure code that evades automated detection mechanisms.

IV. Discussion

Our proposed LLM-based code vulnerability detection framework, using various prompt engineering techniques, showed promising results but also room for improvement. In this section, we discuss some of the key results and research outcomes in more detail.

a) Opportunities for Vulnerability Correction Post-Detection: We explored leveraging LLMs code generation capabilities to correct and patch vulnerable code. We conducted a preliminary code correction experiment using GPT-3.5 and GPT-4o, and the dataset provided by Purple Llama CyberSecEval's instruct benchmark [30]. This dataset was chosen as it included an Insecure Code Detector (ICD) model, fine-tuned to detect vulnerabilities in the dataset with high accuracy. The ICD provides an automated verification tool to test fixes on the Purple Llama benchmark, allowing us to objectively measure whether proposed patches resolve vulnerabilities. By contrast, DiverseVul and many other datasets lack such validation mechanisms, making it impractical to confirm patch correctness without extensive manual review or external tooling. We therefore restricted our patch experiments to Purple Llama.

The experimental process involved using the dataset's provided prompts to generate vulnerable code using GPT-3.5 and GPT-4o. We then used ICD to identify vulnerable codes, which were then used as input to GPT-3.5 for correcting the vulnerable code. The output is a binary (i.e., either fixed or not fixed based on the ICD output). The results shown in Figure 8 show a significant increase in performance brought by GPT-4o, being able to fix 45.77% of the vulnerable codes compared

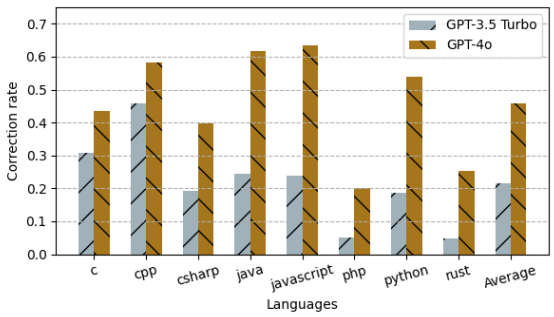


Fig. 8: Preliminary Results of LLM Code Correction

to GPT-3.5, fixing only 21.56% of the vulnerable codes. This indicates that with the correct vulnerability identified, GPT-4o, one of the current best-performing LLMs, can correct such vulnerability at a high correction rate. However, as shown in our experiments, the vulnerability detection performance is poor when considering the automatic approach. Further, the robust verification of a true correction from vulnerable code to benign whilst maintaining the code's intended functionality is non-trivial. This process may require manual validation or automated assessment using custom test harnesses for each specific piece of corrected code. Hence, there is still room for significant improvement in fully automating the vulnerable code detection and correction framework.

b) Dataset Limitations: Our study primarily relies on the DiverseVul dataset [15], which focuses on C/C++ vulnerabilities. While this comprehensive dataset aggregates vulnerabilities from 797 GitHub projects, it may not fully represent vulnerabilities in other programming languages or more recent vulnerability patterns. The dataset's recency (published in 2023) captures the latest vulnerability trends, but the effectiveness of LLM-based vulnerability detection may change over time as new types of vulnerabilities emerge. Our study provides a snapshot of current performance but may not accurately reflect future trends, which limits the generalizability of our findings to other languages and contexts.

c) LLM Selection: We conducted our experiments using GPT-3.5, GPT-4o, Gemini 2.0 Flash, and Meta Llama 3.1, based on our user survey results showing these as preferred models. Thus, our results may not generalize to other LLMs (Claude Sonnet, etc.) or future versions. The lack of access to other models like Microsoft Copilot (preferred by 10.5% of survey respondents) may have limited our comparison scope.

d) Context Definitions: Our choice of context definitions (No Context, Semantic Context, and Full Context) provides a structured approach to vulnerability detection within single-file boundaries. While this aligns with developer workflows and respects LLM limitations, we acknowledge that broader contexts are feasible. The impact of adding multi-file, or repository-wide, contexts remains unclear; however, as LLM capabilities expand, analyzing entire codebases for vulnerabilities becomes increasingly viable. Exploring these expanded context boundaries represents a valuable direction for future research as models evolve to handle larger inputs more effectively.

e) Prompt Engineering Techniques: We explored several prompt engineering strategies based on popular techniques identified in our user survey, including chain-of-thought prompting. However, there may be other effective techniques we did not consider. Our work on vulnerability correction is preliminary and uses a specific dataset (Purple Llama CyberSecEval). A more comprehensive study would be needed to fully assess the correction capabilities of LLMs across various vulnerability types and programming contexts.

f) Vulnerability Detection Metrics: Our classification of vulnerability detection outcomes into five categories (TP, TN, FP, FN, MC) provides a more nuanced view than traditional binary classifications, aligning with the complexity of real-world vulnerability detection. However, this approach may complicate direct comparisons with studies using simpler metrics. Additionally, our preliminary correction experiments only classified outcomes as "Fixed" or "Not fixed," which may not capture the full spectrum of correction quality.

g) Human Factors: The user study results may be influenced by self-selection bias, as participants were recruited through Prolific and required to have both AI chatbot and programming experience. While this ensured relevant expertise, it may not represent the broader population of software developers, potentially affecting the generalizability of our findings on LLM usage patterns and preferences in secure coding practices.

h) Comparisons with Other CVD Approaches: Our focus is on evaluating LLMs for CVD. However, it is important to contextualize these results. Static analysis tools rely on predefined rules, excelling at well-defined patterns but struggling with context-dependent flaws. Prior work shows LLMs can detect vulnerabilities that evade rule-based methods, particularly those involving complex data flow (see Section VI-B). Further, deep learning-based detection methods have been benchmarked on DiverseVul [31], with F1 scores comparable to our LLM performance. However, these models require dataset-specific training and lack LLMs' adaptability. Recent research also explores combining static analysis with LLMs [32], [33]. However, existing hybrid methods do not integrate structured context-awareness as we do. Future work could explore LLM-assisted static analysis with context-awareness to enhance automated vulnerability detection.

i) Ethical and Privacy Considerations: The use of LLMs for vulnerability detection may raise important ethical and privacy concerns. A primary concern is data privacy, as proprietary code submitted to LLMs may be exposed to model providers, posing confidentiality risks [34]. Organizations adopting LLM-based vulnerability detection must consider secure deployment strategies, such as self-hosted models or data anonymization, to minimize the risk of sensitive information leakage. In addition to privacy concerns, the reliability of LLM-generated results must also be addressed. LLMs can introduce outdated dependencies, security flaws, or vulnerable code patterns. Moreover, developers may overly trust AI-generated suggestions, with 91.9% unaware of AI poisoning

risks [35]. To mitigate these risks, developer education is essential to ensure that human reviewers critically assess and refine AI-generated code. Additionally, existing vulnerability analysis tools can complement LLM-assisted detections, improving their accuracy and robustness.

V. Threats to Validity

We summarize potential threats to the validity of our study. **External Validity:** There are two potential threats to the external validity of this study: (1) **Dataset Selection**: The dataset used in our study focuses on C/C++ vulnerabilities, aggregated from 797 GitHub projects. While comprehensive, it may not fully represent vulnerabilities in other programming languages or the most recent vulnerability patterns. Although published in 2023 and capturing the latest trends, the effectiveness of LLM-based vulnerability detection could evolve over time as new vulnerabilities emerge. Thus, our study provides a snapshot of current performance but may not reflect future trends, limiting the generalizability of our findings to other languages and contexts. (2) **Survey Scope**: Our survey did not explicitly capture demographic factors such as participants' specific software development domains, organizational structures, or company sizes. These factors can influence secure coding practices, as security considerations in mission-critical domains may differ significantly from those in general-purpose software development. However, the primary goal of our survey was not to conduct a demographic analysis or create hardlined generalizations but rather to gather broad insights into participants' attitudes and behaviors regarding LLM usage in secure coding contexts. This provided insight into user attitudes, behaviors, and preferences regarding LLM usage in secure coding, including the frequency of LLM use for vulnerability identification and the types of LLMs most commonly adopted, which guided our selection of models, evaluation criteria, and experimental setup.

Construct Validity: There are also two potential threats to the construct validity of this study: (1) **LLM Selection**: As discussed in Section IV-0c, the selection of four LLMs: GPT-3.5, GPT-4, Gemini and Llama may have limited our comparison scope. (2) **Prompt Engineering Techniques**: We explored several prompt engineering strategies based on popular techniques identified in our user survey, including chain-of-thought prompting. However, there may be other effective techniques we did not consider. Our work on vulnerability correction is preliminary and uses a specific dataset (Purple Llama CyberSecEval). A more comprehensive study would be needed to fully assess the correction capabilities of LLMs across various vulnerabilities and programming contexts.

Reliability: The reliability of this study is subject to threats related to the data collection and analysis process. The user study results may be influenced by self-selection bias; participants were recruited through Prolific and required to have experience with AI chatbots and programming. While this ensured relevant expertise, it may not represent the broader population of software developers, potentially affecting the generalizability of our findings on LLM usage patterns and preferences in secure coding practices. To address this threat, we explicitly detailed the data collection and analysis process in this study. Another threat to this study is the metrics used for vulnerability detection. Our classification of detection outcomes into five categories (TP, TN, FP, FN, MC) offers a more nuanced view than traditional binary classifications, reflecting the complexity of real-world vulnerability detection. However, this approach may complicate direct comparisons with studies using simpler metrics. Additionally, our preliminary correction experiments classified outcomes as either 'Fixed' or 'Not fixed,' which may not capture the full spectrum of correction quality.

VI. Related Work

A. Code Vulnerability Detection

Traditional methods for mitigating code-based vulnerabilities rely on a combination of techniques [36]:

- **Manual code review:** Human inspection of source code to identify security flaws. While effective, this method is time-consuming and susceptible to human error [37].
- **Static analysis tools:** Automated analysis of code without execution, searching for patterns indicating potential vulnerabilities. These tools are efficient but may struggle with complex or context-dependent issues [38].
- **Dynamic testing:** Execution of code under controlled conditions to observe behavior and identify runtime vulnerabilities [38].

Recent approaches to CVD have primarily focused on advanced static analysis tools and machine learning (ML) methods [39]–[41]. However, these approaches have shown limitations in real-world applications and out-of-domain settings. Lipp et al. [42] demonstrated that state-of-the-art static C code analyzers miss approximately half of the known vulnerabilities, while Harzevili et al. [43] found that popular static analysis tools detect only 0.01% of known security vulnerabilities in ML libraries.

Applying LLMs to vulnerability detection is an emerging research area with varying results. This aligns with our focus on how LLM versions and code structures affect detection accuracy. Prior work shows LLMs often surpass static analysis tools in security code review, with GPT-4 excelling, especially when guided by a CWE list [44]. Noever [10] found GPT-4 detected four times more vulnerabilities than static analyzers. Lu et al. [45] enhanced LLM-based detection by integrating graph structures and in-context learning, improving F1 scores by at least 28.65% on C/C++ datasets.

However, contrasting results have also been reported. Steenhoek et al. [11] found that LLMs perform worse than humans and struggle with understanding security concepts. Purba et al. [12] noted high false-positive rates in LLM vulnerability detection, though they acknowledged potential for improvement.

The nascent state of LLM-based CVD presents significant opportunities for advancement. Our study aims to address these gaps by exploring various prompt engineering strategies, examining the impact of code context, comparing different

LLM versions, and assessing vulnerability detection capabilities across single- and multi-function code structures.

B. Prompt Engineering for CVD

The advent of LLMs has introduced novel approaches to CVD, particularly through prompt engineering strategies. These strategies aim to improve LLM output quality by guiding their responses through carefully crafted inputs. Current approaches can be broadly categorized into two types [46]:

- **Zero-shot prompting:** LLMs are given a prompt without any examples, typically directly asking if the given code contains vulnerabilities [47], [48].
- **Few-shot prompting:** This involves providing the LLM with examples along with the prompt. One such strategy is retrieval augmentation, which involves retrieving similar labeled data samples to guide the prediction of LLMs on test samples [46].

Liu et al. [49] demonstrated that retrieving relevant code samples and incorporating them as context inputs enhances the code vulnerability detection performance of GPT models.

Advanced prompt engineering techniques have shown promising results. Zhang et al. [50] integrated data flow and API call sequence information into a chain-of-thought prompting strategy for ChatGPT, achieving high accuracy across Java and C/C++ vulnerabilities. Bae et al. [51] showed that tailored prompts substantially improved vulnerability detection performance of GPT-4 and Claude-3.5 Sonnet, with their Step-by-Step prompts yielding impressive F1 scores of 0.9072 and 0.8933, respectively. Khare et al. [52] proposed a dataflow analysis-based prompt strategy, designed to help LLMs detect security vulnerabilities by simulating a source-sink-sanitizer analysis within the prompt. This approach showed significant improvement in identifying specific vulnerability classes that require only intra-procedural reasoning (e.g., OS Command Injection, NULL Pointer Dereference). However, these methods focus on isolated code snippets, which may not fully support real-world applications where developers analyze vulnerabilities in the broader context of interdependent functions or modules. Our approach addresses this gap by incorporating the full code context, which is essential for detecting vulnerabilities across complex, multi-function structures.

C. Evaluation Methods

The CWE serves as a standardized dictionary of software weaknesses and vulnerabilities, providing a common language for describing software security flaws, enabling developers and security professionals to effectively communicate and address issues [53]. In the context of CVD, utilizing CWE classification allows for a more systematic and targeted approach.

Several datasets have been proposed for CVD in codebases of various languages, where each datapoint may describe benign or vulnerable code, the latter typically labeled by CWE. However, research has shown that the suitability of these datasets is not universal. Synthetic datasets like SATE IV Juliet and SARD offer accurate test cases but often fail to capture the complexities of real-world vulnerabilities. In contrast, datasets labeled by static analyzers can be large but suffer from potentially low label accuracy due to the limitations of static analysis tools [54]. Manually labeled datasets like Devign provide high-quality labels but are limited in size due to the labor-intensive labeling process. Devign requires approximately 600 man-hours of expert review [19]. Datasets derived from security issues, such as ReVeal, BigVul, and CVEfixes, are generally considered effective at identifying vulnerability-fixing commits and representative of in-the-wild vulnerabilities but are often limited in scope or diversity [15]. Recently, the DiverseVul dataset [15] has been published in an attempt to address these issues, aiming to provide a larger and more diverse collection of vulnerabilities compared to previous datasets. However, DiverseVul only contains C/C++ based vulnerabilities, limiting its applicability to other programming languages and potentially overlooking language-specific vulnerability patterns.

VII. Conclusion

We propose a framework for code vulnerability detection using LLMs, incorporating advanced prompt engineering strategies to improve detection accuracy. We evaluated several prompting techniques on the DiverseVul dataset of C/C++ vulnerabilities, assessing performance across different code structures, contextual information levels, and LLM capabilities. Our experiments demonstrated that prompt engineering can significantly influence LLM vulnerability detection performance, with our proposed Dynamic method improving F1 scores by up to 100% for GPT-3.5. However, we found that the effects of prompt engineering diminished with more advanced models like GPT-4o, Gemini 2.0 Flash and Meta Llama 3.1.

Importantly, our results indicate that current LLM-based vulnerability detection still faces significant challenges. Performance varied widely across various CWE categories, with examined LLMs showing up to 3.5 times better F1 scores for the top 25 CWEs compared to other vulnerabilities. We also observed that accuracy declined when analyzing larger code contexts; all models struggled with correctly identifying the type of vulnerability in the code, with the best F1 score observed being only 0.16. This highlights the difficulty LLMs face in precise vulnerability classification.

Our work demonstrates the potential of LLMs for code vulnerability detection but also highlights key limitations. Improving detection accuracy, particularly for less common vulnerabilities, remains a challenge. In human-in-the-loop scenarios, both models achieved consistent accuracies of around 50%. Preliminary experiments on vulnerability correction showed a 45.77% success rate using GPT-4o, indicating a promising direction for enhancing software security. As LLMs evolve, ongoing research will be essential to fully leverage their capabilities while addressing associated risks.

Acknowledgment

This work is partially supported by the IITP grant (No. RS-2024-00439762, RS-2024-00451909).

REFERENCES

[1] R. Hackett, "Stagefright: Everything you need to know about google's android megabug." https://fortune.com/2015/07/28/stagefright-google-android-security/, 2015. (accessed: Aug. 02, 2024).

[2] K. Goseva-Popstojanova and A. Perhinschi, "On the capability of static code analysis to detect security vulnerabilities," *Information and Software Technology*, vol. 68, pp. 18–33, 2015.

[3] C. Vassallo, S. Panichella, F. Palomba, S. Proksch, H. C. Gall, and A. Zaidman, "How developers engage with static analysis tools in different contexts," *Empirical Software Engineering*, vol. 25, pp. 1419–1457, 2020.

[4] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?," *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2021.

[5] S. Rajapaksha, J. Senanayake, H. Kalutarage, and M. O. Al-Kadri, "Ai-powered vulnerability detection for secure source code development," in *International Conference on Information Technology and Communications Security*, pp. 275–288, Springer, 2022.

[6] G. Destefanis, S. Bartolucci, and M. Ortu, "A preliminary analysis on the code generation capabilities of gpt-3.5 and bard ai models for java functions," *arXiv preprint arXiv:2305.09402*, 2023.

[7] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 1430–1442, IEEE, 2023.

[8] Y. Zhang *et al.*, "Automatic commit message generation: A critical review and directions for future work," *IEEE Transactions on Software Engineering*, 2024.

[9] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, *et al.*, "Palm: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.

[10] D. Noever, "Can large language models find and fix vulnerable software?," *arXiv preprint arXiv:2308.10345*, 2023.

[11] B. Steenhoek, M. M. Rahman, M. K. Roy, M. S. Alam, E. T. Barr, and W. Le, "A comprehensive study of the capabilities of large language models for vulnerability detection," *arXiv preprint arXiv:2403.17218*, 2024.

[12] M. D. Purba, A. Ghosh, B. J. Radford, and B. Chu, "Software vulnerability detection using large language models," in *2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW)*, pp. 112–119, IEEE, 2023.

[13] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, "Unleashing the potential of prompt engineering in large language models: a comprehensive review," *arXiv preprint arXiv:2310.14735*, 2023.

[14] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Transactions on Software Engineering and Methodology*, 2024.

[15] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*, pp. 654–668, 2023.

[16] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[17] OpenAI, "Hello gpt-4o." https://openai.com/index/hello-gpt-4o/, 2024. (accessed Jun. 06, 2024).

[18] S. Christey, J. Kenderdine, J. Mazella, and B. Miles, "Common weakness enumeration," *Mitre Corporation*, 2013.

[19] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.

[20] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A c/c++ code vulnerability dataset with code changes and cve summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories*, MSR '20, (New York, NY, USA), p. 508–512, Association for Computing Machinery, 2020.

[21] G. Nikitopoulos, K. Dritsa, P. Louridas, and D. Mitropoulos, "Crossvul: a cross-language vulnerability dataset with commit data," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1565–1569, 2021.

[22] G. Bhandari, A. Naseer, and L. Moonen, "Cvefixes: automated collection of vulnerabilities and their fixes from open-source software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, pp. 30–39, 2021.

[23] A. Kong, S. Zhao, H. Chen, Q. Li, Y. Qin, R. Sun, and X. Zhou, "Better zero-shot reasoning with role-play prompting," *arXiv preprint arXiv:2308.07702*, 2023.

[24] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *Advances in neural information processing systems*, vol. 35, pp. 22199–22213, 2022.

[25] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," *arXiv preprint arXiv:2211.01910*, 2022.

[26] S. Teo, "How i won singapore's gpt-4 prompt engineering competition," *Medium*, 2024. Accessed: 2024-04-21.

[27] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, *et al.*, "Dspy: Compiling declarative language model calls into self-improving pipelines," *arXiv preprint arXiv:2310.03714*, 2023.

[28] Y. Shvartzshnaider, V. Duddu, and J. Lacalamita, "Llm-ci: Assessing contextual integrity norms in language models," 2024.

[29] M. Renze and E. Guven, "The effect of sampling temperature on problem solving in large language models," 2024.

[30] M. Bhatt *et al.*, "Purple llama cyberseceval: A secure coding benchmark for language models," *arXiv preprint arXiv:2312.04724*, 2023.

[31] R. Liu, Y. Wang, H. Xu, J. Sun, F. Zhang, P. Li, and Z. Guo, "Vul-lmgnns: Fusing language models and online-distilled graph neural networks for code vulnerability detection," *Information Fusion*, vol. 115, p. 102748, 2025.

[32] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "Assisting static analysis with large language models: A chatgpt experiment," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 2107–2111, 2023.

[33] Z. Li, S. Dutta, and M. Naik, "Llm-assisted static analysis for detecting security vulnerabilities," *arXiv preprint arXiv:2405.17238*, 2024.

[34] "11% of data employees paste into ChatGPT is confidential." https://www.cyberhaven.com/blog/4-2-of-workers-have-pasted-company-data-into-chatgpt. [Online; accessed 2.24.2025].

[35] S. Oh, K. Lee, S. Park, D. Kim, and H. Kim, "Poisoned chatgpt finds work for idle hands: Exploring developers' coding practices with insecure suggestions from poisoned ai models," in *IEEE Symposium on Security and Privacy (SP)*, 2024.

[36] H. Shahriar and M. Zulkernine, "Mitigating program security vulnerabilities: Approaches and challenges," *ACM Computing Surveys (CSUR)*, vol. 44, no. 3, pp. 1–46, 2012.

[37] H. Hanif, M. H. N. M. Nasir, M. F. Ab Razak, A. Firdaus, and N. B. Anuar, "The rise of software vulnerability: Taxonomy of software vulnerabilities detection and machine learning approaches," *Journal of Network and Computer Applications*, vol. 179, p. 103009, 2021.

[38] A. Kaur and R. Nayyar, "A comparative study of static code analysis tools for vulnerability detection in c/c++ and java source code," *Procedia Computer Science*, vol. 171, pp. 2023–2029, 2020.

[39] A. C. Eberendu, V. I. Udegbe, E. O. Ezennorom, A. C. Ibegbulam, T. I. Chinebu, *et al.*, "A systematic literature review of software vulnerability detection," *European Journal of Computer Science and Information Technology*, vol. 10, no. 1, pp. 23–37, 2022.

[40] Y. Bi, J. Huang, P. Liu, and L. Wang, "Benchmarking software vulnerability detection techniques: A survey," *arXiv preprint arXiv:2303.16362*, 2023.

[41] H. Baek, M. Lee, and H. Kim, "CryptoLLM: Harnessing the Power of LLMs to Detect Cryptographic API Misuse," in *Proceedings of the 29th European Symposium on Research in Computer Security (ESORICS)*, 2024.

[42] S. Lipp, S. Banescu, and A. Pretschner, "An empirical study on the effectiveness of static c code analyzers for vulnerability detection," in *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*, pp. 544–555, 2022.

[43] N. S. Harzevili, J. Shin, J. Wang, S. Wang, and N. Nagappan, "Automatic static vulnerability detection for machine learning libraries: Are we there yet?," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pp. 795–806, IEEE, 2023.

[44] J. Yu, P. Liang, Y. Fu, A. Tahir, M. Shahin, C. Wang, and Y. Cai, "Security code review by llms: A deep dive into responses," *arXiv preprint arXiv:2401.16310*, 2024.

[45] G. Lu, X. Ju, X. Chen, W. Pei, and Z. Cai, "Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning," *Journal of Systems and Software*, vol. 212, p. 112031, 2024.

[46] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and roadmap," *arXiv preprint arXiv:2404.02525*, 2024.

[47] M. Fu, C. K. Tantithamthavorn, V. Nguyen, and T. Le, "Chatgpt for vulnerability detection, classification, and repair: How far are we?," in *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*, pp. 632–636, IEEE, 2023.

[48] X. Zhou, T. Zhang, and D. Lo, "Large language model for vulnerability detection: Emerging results and future directions," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, pp. 47–51, 2024.

[49] Z. Liu, Q. Liao, W. Gu, and C. Gao, "Software vulnerability detection with gpt and in-context learning," in *2023 8th International Conference on Data Science in Cyberspace (DSC)*, pp. 229–236, IEEE, 2023.

[50] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, and H. Li, "Prompt-enhanced software vulnerability detection using chatgpt," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, pp. 276–277, 2024.

[51] J. Bae, S. Kwon, and S. Myeong, "Enhancing software code vulnerability detection using gpt-4o and claude-3.5 sonnet: A study on prompt engineering techniques," *Electronics*, vol. 13, no. 13, p. 2657, 2024.

[52] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," *arXiv preprint arXiv:2311.16169*, 2023.

[53] MITRE, "Cwe - common weakness enumeration." https://cwe.mitre.org/, 2024.

[54] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 121–133, IEEE, 2023.